

第13章 模块与工程化

import 机制 · sys.path · venv 与 pip · pyproject.toml

本章目标:讲透 Python 的模块系统与工程化:import 的搜索与缓存机制、包与 `__init__.py`、绝对/相对导入、循环导入陷阱,再到 venv 虚拟环境、pip 依赖管理、pyproject.toml 项目布局与 PyPI 发布。学完你能组织出一个多文件、可测试、可打包、可交付的规范项目。

直觉起点:C 的 `#include` 在编译期把头文件文本粘贴进来,链接期解决符号;Python 的 `import` 在运行期执行"模块文件本身",并缓存到 `sys.modules`。模块即"执行一次的文件",包即"带 `__init__.py` 的目录"——理解这两句话,import 的一切行为都有了解释。

前置要求:第4章函数与作用域、第11章动态导入(importlib)、第12章装饰器(项目常配装饰器库)。

本章路线:第一阶段(1.5 小时)读 §13.1–§13.4,理解 import 机制;第二阶段(1.5 小时)读 §13.5–§13.6,掌握 venv 与 pip;第三阶段(2 小时)读 §13.7–§13.8,完成项目布局与章末实验。

本章知识导图

- └─ 13.1 import 机制 — 执行一次、sys.modules 缓存
- └─ 13.2 sys.path 与包 — 搜索路径与 `__init__.py`
- └─ 13.3 导入风格 — 绝对导入与相对导入
- └─ 13.4 `__name__` 守卫 — 脚本与模块双模式
- └─ 13.5 venv — 虚拟环境隔离依赖
- └─ 13.6 pip — 安装、锁定与升级
- └─ 13.7 pyproject.toml — 现代项目布局
- └─ 13.8 发布与交付 — PyPI 与版本语义

§13.1 import 机制:执行一次,缓存永久

`import json` 在运行期做的事:① 在 `sys.path` 里找 `json.py`;② 首次导入时执行整个文件(模块顶层代码全部运行);③ 把结果放进 `sys.modules["json"]` 缓存;④ 以后每次 `import` 直接取缓存,不再执行。因此"模块顶层代码只执行一次",副作用(注册、打印)也只发生一次:

```
# db.py — 一个模块文件
print("db.py 被导入")                # 整个项目只打印一次

CONNECTIONS = []                     # 模块级共享状态

def connect(name: str) -> None:
    CONNECTIONS.append(name)

# main.py — 另一个文件
import db                             # 执行 db.py, 缓存
import db                             # 直接命中缓存, 不执行
db.connect("orders")
print(db.CONNECTIONS)                # ['orders']
```

C(#include)	Python(import)
编译期文本粘贴,重复包含要 #pragma once	运行期执行一次,缓存复用,天然防重
头文件与实现分离(.h/.c)	一个 .py 既是声明也是实现
编译单元间靠链接器结合	import 即链接,依赖关系运行时可见
头文件变化触发全量重编译	模块变化只需重跑程序

模块级共享状态(CONNECTIONS)是“模块即单例”的体现:任何 import 它的文件看到的是同一个列表。生产上常利用这点做配置中心、连接池、注册表——但注意线程安全(第9章)。`import db` 后访问 `db.xxx`,与 `from db import connect` 的区别:后者把名字直接绑进当前命名空间,改的是“副本绑定”,模块里的原对象不受影响。

与 C 的头文件不同,import 建立的是运行时对象关系:db.CONNECTIONS 是同一个列表对象,任何一处 append,全项目可见。这个特性让“模块级单例”成为 Python 组织共享状态的主要方式——配置、连接池、注册表都习惯放在模块顶层。生产提醒:模块级可变状态要与第9章线程安全结合起来考虑,必要时加锁或用不可变数据。

§13.2 sys.path 与包: __init__.py 的职责

import 在 `sys.path` 的目录列表里按顺序找模块:当前脚本目录、环境变量 PYTHONPATH、标准库、site-packages。把 import 失败当线索:先 `print(sys.path)` 看搜索路径,再看模块是否真的躺在某个路径下:

```
import sys
print(sys.path)
# ['项目根', ..., 'python312\\Lib\\site-packages', ...]

# 包 = 带 __init__.py 的目录;包内模块用点号路径导入
# project/
#   app/
#     __init__.py    ← 有此文件,app 才是包
#     main.py
#     utils/
#       __init__.py
#       format.py
import app.utils.format as fmt    # 点号路径即目录层级
```

`__init__.py` 有两个职责:① 标记目录为包(没有它,同名的普通目录不会被当作包导入);② 收纳“对外 API”——在 `__init__.py` 里 `from .utils.format import fmt_time`,使用方就能 `from app import fmt_time`,内部结构调整不影响调用方。这是包设计的核心手法:内部自由重组,对外保持稳定。

判断一个目录是否真是包,就看 `__init__.py` 是否存在。Python 3.3 引入的命名空间包允许无 `__init__.py` 的目录被导入,但那只是隐式机制;生产项目一律显式写 `__init__.py`,把“包边界”写清楚——既能收敛 API,也能防止两个同名模块被意外合并。

§13.3 导入风格:绝对导入与相对导入

绝对导入从包根写全路径(`from app.utils import format`),清晰但写起来长;**相对导入**从当前包出发(`from .utils import format`、`from ..common import x`),重构时跟随包移动。生产规范:包内互引用一律相对导入,包外一律绝对导入:

```
# app/utils/format.py
from ..common.constants import MAX_LEN  # 相对导入:上一级包的模块

# app/common/constants.py
MAX_LEN = 100

# 顶层入口 app/main.py
from app.utils.format import fmt_time  # 绝对导入(以包根为锚)
```

误区:循环导入。 a.py 顶部 import b,而 b.py 顶部 import a——a 还没执行完就回头找 b,而 b 正在等 a,必然 ImportError(或拿到半初始化模块)。常见诱因:把“运行时才需要的导入”提到模块顶部、两个模块互相依赖对方常量。解法:① 依赖方向理成单向(公共代码下沉到 common);② 把导入挪进函数体内(延迟到调用时,此时两边都已加载完);③ 用 `__init__.py` 做组装层解耦。

相对导入的边界:被 `python -m` 运行或被导入时,相对导入才有效;直接 `python app/utils/format.py` 运行会报“attempted relative import”。所以包内脚本一律从项目根用 `python -m app.main` 启动——这也是工程化第一步。

§13.4 `__name__` 守卫:脚本与模块双模式

一个 .py 文件被直接运行时 `__name__ == "__main__"`,被导入时 `__name__ == "模块路径"`。利用这一点,同一个文件既能当脚本跑,也能被导入复用,且导入时不会误执行演示代码:

```
"""订单工具:可导入,也可命令行直接运行"""

def total(prices: list) -> float:
    return sum(prices)

if __name__ == "__main__":
    # 只有直接运行时才执行;被 import 时跳过
    print(total([1.0, 2.5, 3.0]))  # 6.5
```

这是工程化的基础纪律:所有可执行入口文件都包 `__main__` 守卫,并习惯用 `python -m` 运行包内模块(它会把包根放进 `sys.path`,相对导入可用)。第9章的进程池、第8章的服务器入口都依赖这个守卫,忘了写会在导入时触发整段逻辑。

一个常见反例:工具脚本在文件末尾直接写测试代码,被他人 import 时测试逻辑全量执行,轻则打印污染输出,重则执行删除等危险操作。养成习惯:模块文件的顶层只放定义与常量,一切“运行时才该发生的事”放进函数,再由 `__main__` 守卫决定是否调用。

§13.5 venv:虚拟环境隔离依赖

不同项目需要不同版本的第三方库,装进系统全局会互相打架——**venv** 为每个项目建独立环境:自己的 python、自己的 site-packages。创建与激活:

```
# 创建虚拟环境(在项目根,一次)
python -m venv .venv

# 激活(每次新终端)
# Windows(CMD): .venv\Scripts\activate
# Windows(PowerShell): .venv\Scripts\Activate.ps1
# Linux/macOS: source .venv/bin/activate

# 激活后,终端提示符出现 (.venv),pip 安装全部进入本项目
python -m pip install requests
python -c "import requests; print(requests.__version__)"
```

C(系统库)	Python(venv)
库装进系统,版本全局唯一	每项目独立 site-packages,版本互不干扰
升级系统库影响所有程序	升级只影响当前虚拟环境
依赖冲突只能靠改名/静态链接	环境隔离从源头消除冲突
部署要匹配目标机库版本	requirements 锁定 + 重建环境,可复现

误区:依赖装进了全局。忘了激活虚拟环境就 pip install,包落进系统 Python——换个环境程序就崩,排查时"我明明装了"却 ImportError。生产规范:每个项目第一件事创建 .venv 并激活;pip 一律用 `python -m pip` (避免"系统 pip 与当前 python 不一致");.venv 目录加入 .gitignore 绝不提交。

§13.6 pip:安装、锁定与升级

pip 的日常四件事:安装、锁定、按锁定安装、升级。锁定用 `pip freeze > requirements.txt` 导出当前环境全部依赖的精确版本,换机器 `pip install -r requirements.txt` 一键复现:

```
# 安装指定版本与范围
python -m pip install requests==2.31.0
python -m pip install "flask>=3.0,<4"      # 版本范围

# 锁定当前环境(记录精确版本)
python -m pip freeze > requirements.txt

# 按锁定文件重建环境
python -m pip install -r requirements.txt

# 升级 pip 自身与某依赖
python -m pip install --upgrade pip
python -m pip install --upgrade requests
```

误区:requirements.txt 不锁版本或随手删。写 `requests` 不写版本,今天装 2.31、下周装 3.x,行为漂移;删掉某行,别人复现环境时报错也查不出。规范:requirements.txt 由 freeze 生成、含精确版本、随代码入库;生产环境锁"平台无关的纯版本",必要处注释说明约束原因(如 "flask==3.0.3 # 4.x 改 API")。

升级依赖后的第一动作是跑全量测试;升级大版本(3.x → 4.x)前先读 release notes,别让 pip 替你决定命运。锁版本的精确度要分层:开发用范围,部署用精确——第14章会讲配套的版本策略。

§13.7 pyproject.toml:现代项目布局

Python 3.11+ 的现代项目用 **pyproject.toml** 声明一切:元数据、依赖、构建系统、工具配置。推荐布局——src 布局把"安装后的包"与"源码"分开,杜绝"测试能过、装完就崩"的经典事故:

```
# 项目布局(名字假设为 orderlib)
# orderlib/
#   pyproject.toml
#   README.md
#   src/
#       orderlib/
#           __init__.py
#           pricing.py
#           checkout.py
#   tests/
#       test_pricing.py
#       test_checkout.py

# pyproject.toml 核心内容
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "orderlib"
version = "0.1.0"
description = "订单计算库"
requires-python = ">=3.10"
dependencies = ["requests>=2.31"]

[project.optional-dependencies]
dev = ["pytest", "mypy", "ruff"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

src 布局的收益:测试从项目根运行时 import 到的是"安装后的 orderlib"而不是源码目录——安装行为与开发行为一致;构建工具只打包 src/orderlib,不会把 tests、文档意外塞进发行物。 `pip install -e ".[dev]"` 可编辑安装项目本体并带上开发工具,是本地开发的标配命令。

§13.8 发布与交付:PyPI 与版本语义

让项目可被 `pip install` 安装,是交付的最后一公里。构建 sdist/轮子后上传到 PyPI(公共)或私有源(企业):

```
# 构建发行物(项目根)
python -m pip install --upgrade build
python -m build # 生成 dist/*.tar.gz 与 *.whl

# 检查与上传(需要 PyPI 账号与令牌)
python -m pip install --upgrade twine
python -m twine check dist/* # 校验元数据与文件名
python -m twine upload dist/* # 上传 PyPI

# 其他人安装
python -m pip install orderlib
```

C(库分发)	Python(轮子)
源码包 + 编译,目标机要匹配编译器	wheel 是预打包格式,平台无关的纯 Python 轮子直接装
头文件/静态库/动态库三种形态	sdist 源码 + wheel 二进制(纯 Python 或含 C 扩展)
make install 手动管理	pip 自动解析依赖与版本,一键安装
ABI 不兼容要重新编译	cp310 等标签明确 Python 版本兼容范围

版本号用语义化版本(MAJOR.MINOR.PATCH):破坏性变更升 MAJOR,加功能升 MINOR,修 bug 升 PATCH。发布前过一遍质量清单:测试全绿、mypy 干净、README 更新、CHANGELOG 记录、版本号已升——**发布是不可逆操作,宁可多检查十分钟,不要发一个坏版本让全员踩坑。**

工程建议 1:新项目从第一天就用 pyproject.toml + src 布局 + venv + requirements 锁定,别等代码多了再迁移——迁移成本随文件数线性增长,而"规范骨架"只需十分钟。

工程建议 2:CI(持续集成)里加一个"干净环境安装"步骤:全新 venv + pip install -r requirements.txt + 跑测试,专门抓"本地能跑、别人装不上"的问题。这是工程化投入产出比最高的一个检查。

本章速查表

场景	写法
导入模块	<code>import db / from db import connect</code>
导入一次	<code>sys.modules</code> 缓存,重复 <code>import</code> 不重执行
看搜索路径	<code>import sys; print(sys.path)</code>
包内互引	相对导入 <code>from .utils import fmt</code>
包外引用	绝对导入 <code>from app.utils import fmt</code>
脚本/模块双模式	<code>if __name__ == "__main__":</code> 守卫
运行包内模块	<code>python -m app.main</code> (相对导入可用)
建虚拟环境	<code>python -m venv .venv</code> + 激活
装依赖	<code>python -m pip install requests==2.31.0</code>
锁定依赖	<code>pip freeze > requirements.txt</code>
复现环境	<code>pip install -r requirements.txt</code>
项目元数据	pyproject.toml 的 [project] 段
开发安装	<code>pip install -e ".[dev]"</code>
构建发行物	<code>python -m build</code> ,产物在 dist/
上传 PyPI	<code>python -m twine upload dist/*</code>
版本规则	语义化版本:MAJOR.MINOR.PATCH

最小必会清单

1. 能解释 `import` 的三步机制与 `sys.modules` 缓存的作用
2. 能说出 `__init__.py` 的两个职责,并组织一个三层包结构
3. 能区分绝对/相对导入的适用场景,并识别与修复循环导入
4. 能解释 `__name__ == "__main__"` 的语义并正确使用守卫
5. 能创建并激活 `venv`,用 `pip` 安装与锁定依赖
6. 能搭出 `pyproject.toml` + `src` 布局的规范项目,并说出构建发布的流程

自测题

Q

Q1. `import` 一个模块时,为什么顶层 `print` 只执行一次?强制重新执行怎么做?

答案:首次导入执行模块文件后,结果缓存进 `sys.modules`,后续 `import` 直接取缓存。强制重载:`importlib.reload(module)`——但 `reload` 不改写已绑定的引用(`from` 导入的名字不会更新),生产上几乎不用,调试热更新偶尔用。

Q

Q2. 循环导入为什么会失败?给出一个生产中最常见的诱因与两种解法。

答案:A 顶部 import B 时 B 还没执行完,而 B 顶部又在等 A——一方拿到未初始化模块即失败。常见诱因:两个模块互相引用对方的常量/函数。解法:① 把公共依赖下沉到第三模块,依赖变单向;② 延迟导入——把 import 移进函数体内,调用时才执行,此时两边模块都已加载完。

Q

Q3. 直接运行 `python app/utils/format.py` 报 "attempted relative import",为什么?正确做法?

答案:直接运行时,Python 把该文件当作顶层脚本, `__package__` 为空,相对导入(点号路径)没有锚点。正确做法:从项目根运行 `python -m app.utils.format`,此时解释器按包语义加载,相对导入成立。因此工程规范是"包内一切入口用 `-m` 运行"。

Q

Q4. `venv` 解决了什么问题?为什么说 "pip install 装进了全局" 是常见事故?

答案:`venv` 为项目提供独立 Python 与 `site-packages`,解决多项目依赖版本冲突。未激活 `venv` 时 `pip` 装进系统 Python,换环境(部署机、同事机器)程序就 `ImportError`,且系统环境被污染、权限与安全风险并存。规范:先激活再安装,永远 `python -m pip`,环境信息随项目文档记录。

Q

Q5. `requirements.txt` 为什么不锁版本会 "行为漂移"?`lock` 文件与范围约束各用在哪?

答案:不锁版本时,`pip` 按当前最新解析,今天装 2.x、下周同一命令装 3.x——测试过的组合被悄悄替换,行为漂移且难复现。策略:开发依赖用范围(`flask>=3,<4`)保留升级空间;部署与 CI 用精确锁定(`pip freeze` 或 `lock` 文件),保证 "测试过的版本=线上版本"。

Q

Q6. `src` 布局比 "包直接在项目根" 好在哪?为什么它能防 "测试过、装不上"?

答案:根布局下,测试 `import` 到的是源码目录,而安装后包的路径/内容可能不同(漏包、包含测试文件),出现 "测试全绿、生产崩" 的落差。`src` 布局让测试必须经安装后的路径 `import`,安装行为即测试行为;同时构建只打包 `src` 下的包,发行物干净。代价是安装前无法直接 `import`,属刻意设计。

章末实验题:多文件订单项目

实验 13.1 从零组织可交付项目(覆盖:§13.1–§13.8)

需求:① 按 `src` 布局建项目 `orderlib:src/orderlib/{__init__.py, pricing.py, checkout.py},tests/test_pricing.py`;② `pricing.py` 提供计算逻辑(折扣、税费、总额),`checkout.py` 依赖 `pricing` 并做结算与日志,二者用相对导入;③ `__init__.py` 导出对外 API(`pricing` 与 `checkout` 的核心函数),保证 `from orderlib import calc_total` 可用;④ 所有入口包 `__main__` 守卫,支持 `python -m orderlib.checkout` 直接运行演示;⑤ 写 `pytest` 测试(折扣边界、税额、空订单);⑥ 创建 `venv`、安装 `pytest` 与可编辑安装,跑通测试;⑦ 编写 `pyproject.toml`(元数据、dependencies、dev 依赖、`pytest` 配置),验证 `python -m build` 能产出发行物。

覆盖知识点:import 与包 (§13.1、§13.2)/相对导入与 __name__ 守卫 (§13.3、§13.4)/venv 与 pip (§13.5、§13.6)/pyproject 与构建 (§13.7、§13.8)。

实验环境:Python 3.10+,可联网安装 pytest。

```
# src/orderlib/pricing.py — 计价逻辑(无外部依赖,纯函数)
"""折扣、税额与总额计算"""

DISCOUNT_THRESHOLD = 1000.0
DISCOUNT_RATE = 0.9          # 满 1000 打九折
TAX_RATE = 0.06

def apply_discount(subtotal: float) -> float:
    """满额打折;负值入参直接抛错(入口校验)"""
    if subtotal < 0:
        raise ValueError(f"小计不能为负: {subtotal}")
    if subtotal >= DISCOUNT_THRESHOLD:
        return subtotal * DISCOUNT_RATE
    return subtotal

def calc_tax(amount: float) -> float:
    """按税率计算税额,结果保留两位小数"""
    return round(amount * TAX_RATE, 2)

def calc_total(prices: list) -> dict:
    """入参为单价列表,返回完整账单结构"""
    subtotal = sum(prices)
    discounted = apply_discount(subtotal)
    tax = calc_tax(discounted)
    return {
        "subtotal": round(subtotal, 2),
        "discounted": round(discounted, 2),
        "tax": tax,
        "total": round(discounted + tax, 2),
    }
```

```

# src/orderlib/checkout.py — 结算流程(消费 pricing,演示相对导入)
"""结算入口:组装账单并打印明细"""
from .pricing import calc_total

def checkout(items: list) -> dict:
    """结算:items 为商品对象列表,取 price 字段计价"""
    bill = calc_total([item["price"] for item in items])
    print(f"商品数 {len(items)} | 小计 {bill['subtotal']} | "
          f"折后 {bill['discounted']} | 税 {bill['tax']} | "
          f"总计 {bill['total']}")
    return bill

def main() -> None:
    goods = [{"name": "键盘", "price": 599},
              {"name": "鼠标", "price": 499}]
    checkout(goods)

if __name__ == "__main__":
    main()                                # python -m orderlib.checkout 运行

# src/orderlib/__init__.py — 对外 API 组装
"""orderlib:订单计价与结算库"""
from .pricing import calc_total, calc_tax
from .checkout import checkout

__all__ = ["calc_total", "calc_tax", "checkout"]

# tests/test_pricing.py — 单元测试
import pytest

from orderlib import calc_total

def test_discount_threshold() -> None:
    bill = calc_total([600, 500])          # 1100 触发九折
    assert bill["discounted"] == 990.0

def test_no_discount_below_threshold() -> None:
    bill = calc_total([100, 200])
    assert bill["discounted"] == 300.0

def test_empty_order() -> None:
    bill = calc_total([])
    assert bill["total"] == 0.0

def test_negative_price_rejected() -> None:

```

```
with pytest.raises(ValueError):  
    calc_total([-1])
```

工程清单:① pyproject.toml 按 §13.7 编写, name=orderlib、requires-python=>=3.10、[project.optional-dependencies] dev=["pytest"];② `python -m venv .venv` 并激活;③ `python -m pip install -e ".[dev]"` 可编辑安装;④ `python -m pytest` 全绿;⑤ `python -m build` 产出 dist 下两个发行物;⑥ `python -m pip install dist/orderlib-0.1.0-py3-none-any.whl` 装进干净环境验证"安装即用"。逐条核对:相对导入(是)、__main__ 守卫(是)、__init__ 收敛 API(是)、src 布局(是)、锁定与测试(是)。

下一章预告:第14章 类型系统进阶与生产实践——类型推导与检查、深浅拷贝、None 与 Optional、Union 与 match、dataclass 深度、静态与动态的平衡。全书收官的最后一讲。